24
PARTIAL BOUNDARIES

Full-fledged architectural boundaries are expensive. They require reciprocal polymorphic `Boundary` interfaces, `Input` and `Output` data structures, and all of the dependency management necessary to isolate the two sides into independently compilable and deployable components. That takes a lot of work. It's also a lot of work to maintain.

In many situations, a good architect might judge that the expense of such a boundary is too high—but might still want to hold a place for such a boundary in case it is needed later.

This kind of anticipatory design is often frowned upon by many in the Agile community as a violation of YAGNI: "You Aren't Going to Need It." Architects, however, sometimes look at the problem and think, "Yeah, but I might." In that case,

they may implement a partial boundary.

SKIP THE LAST STEP

One way to construct a partial boundary is to do all the work necessary to create independently compilable and deployable components, and then simply keep them together in the same component. The reciprocal interfaces are there, the input/output data structures are there, and everything is all set up—but we compile and deploy all of them as a single component.

Obviously, this kind of partial boundary requires the same amount of code and preparatory design work as a full boundary. However, it does not require the administration of multiple components. There's no version number tracking or release management burden. That difference should not be taken lightly.

This was the early strategy behind `FitNesse`. The web server component of `FitNesse` was designed to be separable from the wiki and testing part of `FitNesse`. The idea was that we might want to create other web-based applications by using that web component. At the same, we did not want users to have to download two components. Recall that one of our design goals was "*download and go*." It was our intent that users would download one jar file and execute it without having to hunt for other jar files, work out version compatibilities, and so on.

The story of `FitNesse` also points out one of the dangers of this approach. Over time, as it became clear that there would never be a need for a separate web component, the separation between the web component and the wiki component began to weaken. Dependencies started to cross the line in the wrong direction. Nowadays, it would be something of a chore to re-separate them.

ONE-DIMENSIONAL BOUNDARIES

The full-fledged architectural boundary uses reciprocal boundary interfaces to maintain isolation in both directions. Maintaining separation in both directions is expensive both in initial setup and in ongoing maintenance.

A simpler structure that serves to hold the place for later extension to a full-fledged boundary is shown in Figure 24.1. It exemplifies the traditional *Strategy* pattern. A `ServiceBoundary` interface is used by clients and implemented by `ServiceImpl`

classes.

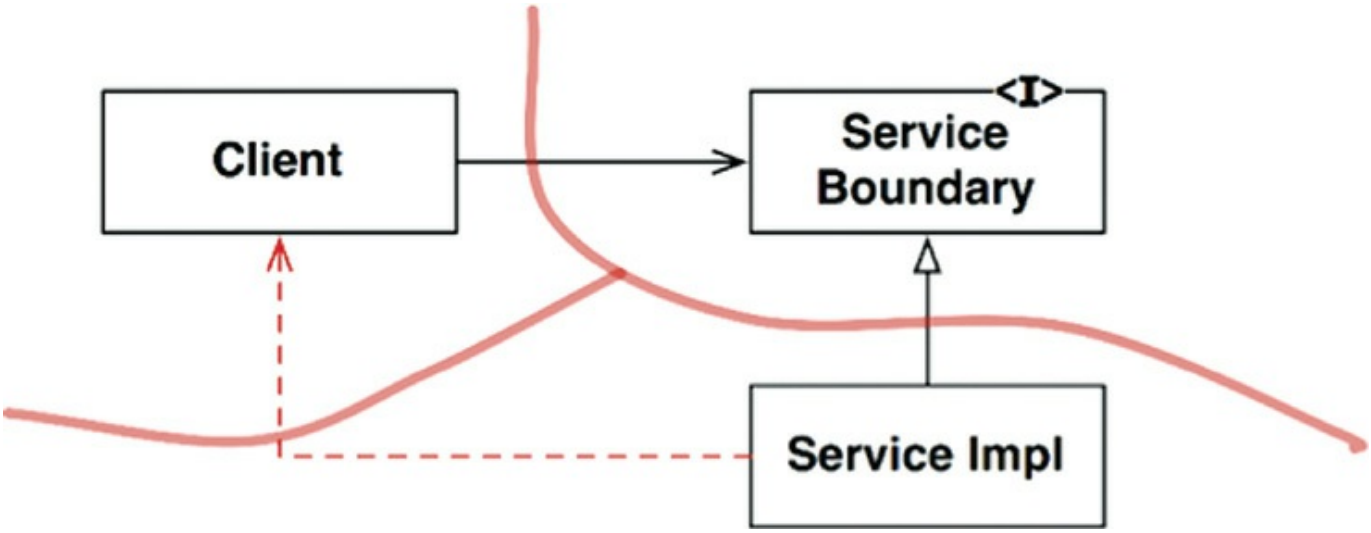

Figure 24.1 The Strategy pattern

It should be clear that this sets the stage for a future architectural boundary. The necessary dependency inversion is in place in an attempt to isolate the `Client` from the `ServiceImpl`. It should also be clear that the separation can degrade pretty rapidly, as shown by the nasty dotted arrow in the diagram. Without reciprocal interfaces, nothing prevents this kind of backchannel other than the diligence and discipline of the developers and architects.

FACADES

An even simpler boundary is the *Facade* pattern, illustrated in Figure 24.2. In this case, even the dependency inversion is sacrificed. The boundary is simply defined by the `Facade` class, which lists all the services as methods, and deploys the service calls to classes that the client is not supposed to access.

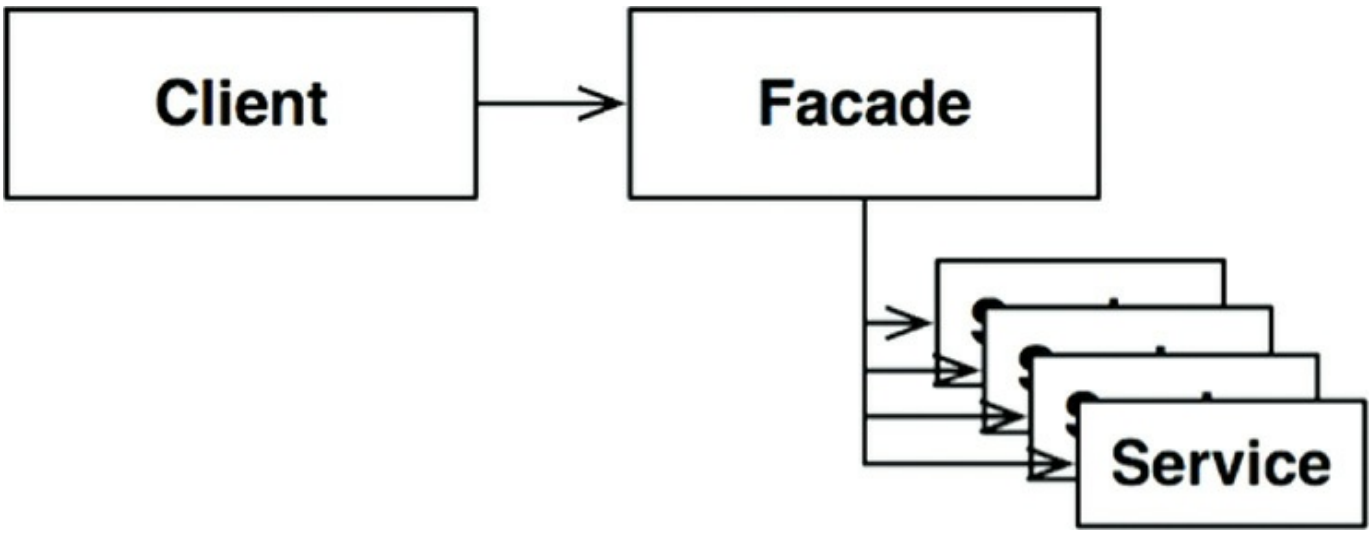

Figure 24.2 The Facade pattern

Note, however, that the `Client` has a transitive dependency on all those service classes. In static languages, a change to the source code in one of the `Service` classes will force the `Client` to recompile. Also, you can imagine how easy backchannels are to create with this structure.

CONCLUSION

We've seen three simple ways to partially implement an architectural boundary. There are, of course, many others. These three strategies are simply offered as examples.

Each of these approaches has its own set of costs and benefits. Each is appropriate, in certain contexts, as a placeholder for an eventual full-fledged boundary. Each can also be degraded if that boundary never materializes.

It is one of the functions of an architect to decide where an architectural boundary might one day exist, and whether to fully or partially implement that boundary.

25
LAYERS AND BOUNDARIES

It is easy to think of systems as being composed of three components: UI, business rules, and database. For some simple systems, this is sufficient. For most systems, though, the number of components is larger than that.

Consider, for example, a simple computer game. It is easy to imagine the three components. The UI handles all messages from the player to the game rules. The game rules store the state of the game in some kind of persistent data structure. But is that all there is?

HUNT THE WUMPUS

Let’s put some flesh on these bones. Let’s assume that the game is the venerable

Hunt the Wumpus adventure game from 1972. This text-based game uses very simple commands like GO EAST and SHOOT WEST. The player enters a command, and the computer responds with what the player sees, smells, hears, and experiences. The player is hunting for a Wumpus in a system of caverns, and must avoid traps, pits, and other dangers lying in wait. If you are interested, the rules of the game are easy to find on the web.

Let's assume that we'll keep the text-based UI, but decouple it from the game rules so that our version can use different languages in different markets. The game rules will communicate with the UI component using a language-independent API, and the UI will translate the API into the appropriate human language.

If the source code dependencies are properly managed, as shown in Figure 25.1, then any number of UI components can reuse the same game rules. The game rules do not know, nor do they care, which human language is being used.

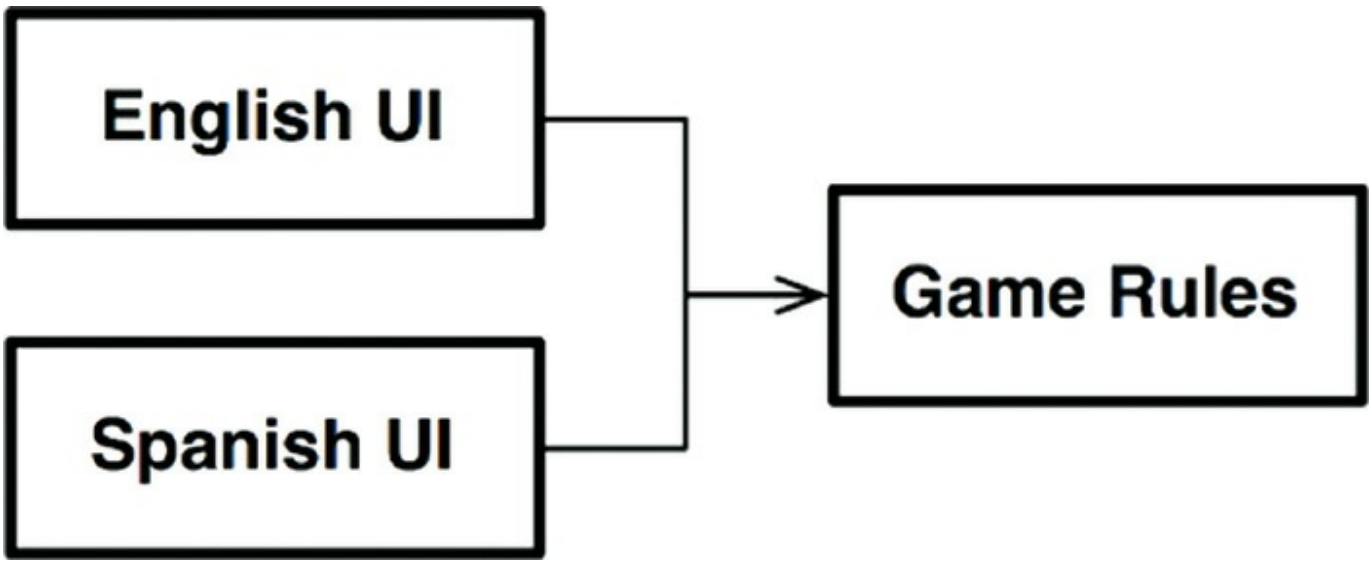

Figure 25.1 Any number of UI components can reuse the game rules

Let's also assume that the state of the game is maintained on some persistent store—perhaps in flash, or perhaps in the cloud, or maybe just in RAM. In any of those cases, we don't want the game rules to know the details. So, again, we'll create an API that the game rules can use to communicate with the data storage component.

We don't want the game rules to know anything about the different kinds of data storage, so the dependencies have to be properly directed following the Dependency Rule, as shown in Figure 25.2.